

☰

 gela2008 / system_dev ▾

🔍

+

📁



Code

Issues

Pull requests

Actions

Projects

Wiki

More ▾

📄

🔗 master ▾

system_dev / system / 詳細設計書.md

📄

⋮

 gela2008 書類にメンバー情報を追加

8f47e83 · 2 hours ago

🕒

204 lines (159 loc) · 8.16 KB



Preview

Code

Blame

Raw

📄

⬇

✎

▾

☰

詳細設計書

メンバー

名前	GitHub ID	学籍番号
戸田陽菜	gela2008	i23027

実装リポジトリ : https://github.com/gela2008/system_dev

1. モジュール一覧

本システムでは

- 画面表示
 - 認証
 - Todo管理
 - データベース管理
- 等の機能を複数のPHPモジュールに分割して実装する

1.1 システム機能

モジュール	ファイル	入力	出力
DB接続	db.php	DBファイル	PDO接続

モジュール	ファイル	入力	出力
認証	auth.php	ユーザー名、パスワード	認証結果
ログイン	login.php	ユーザー名、パスワード	ホーム画面またはエラー
ログアウト	logout.php	セッション	ログイン画面
ホーム表示	home.php	セッション	ホーム画面
カレンダー表示	index.php	年、月、セッション	カレンダー画面
日付別一覧表示	date.php	日付、セッション	Todo一覧
Todo追加	add.php	Todo入力情報	DB登録、一覧画面
詳細表示	detail.php	Todo ID、セッション	詳細画面
Todo編集	edit.php	Todo ID、編集情報	DB更新、詳細画面
Todo削除	delete.php	Todo ID	DB削除、一覧画面
完了状態変更	complete.php	Todo ID、完了状態	DB更新
お知らせ表示	notice.php	セッション、日付	お知らせ一覧

1.2 初期設定・管理用モジュール

モジュール	ファイル	役割
DB初期構築	setup_db.php	テーブルを作成する
科目初期登録	insert_subjects.php	科目データを登録する
ユーザー初期登録	insert_user.php	ユーザーデータを登録する
ユーザー登録	register.php	ユーザーを登録する
DB確認	check_db.php	DB内のデータを確認する
Todo確認	check_todos.php	Todoデータを確認する
スキーマ確認	show_schema.php	DBのテーブル構造を確認する

2. ログイン機能

2.1 処理フロー

1. ユーザーがログイン画面でユーザー名とパスワードを入力する。
2. login.php がフォームから入力値を受け取る。
3. 入力されたユーザー名とパスワードを auth.php に渡す。
4. auth.php が user テーブルからユーザー名に一致するユーザーを検索する。
5. 取得したユーザー情報と入力されたパスワードを照合する。
6. パスワードが一致する場合、ログイン成功とする。
 - セッションに student_id を保存する。
 - home.php へリダイレクトする。
7. ユーザーが存在しない、またはパスワードが一致しない場合、ログイン失敗とする。
 - エラーメッセージを表示する。
 - ログイン画面に留まる。

2.2 分岐条件

条件	処理
ユーザー名とパスワードが一致	student_id をセッションに保存し、home.php へ移動
ユーザーが存在しない	エラーメッセージを表示
パスワードが一致しない	エラーメッセージを表示

2.3 モジュール入出力

モジュール	入力	出力
login.php	ユーザー名、パスワード	認証結果
auth.php	ユーザー名、パスワード	ユーザー情報または認証失敗

3. Todo追加機能

3.1 処理フロー

1. ユーザーが追加画面でタスクまたはスケジュールを選択する。
2. add.php が入力されたTodo情報を受け取る。
3. 入力されたタイトルを確認する。

4. タイトルが空の場合、エラーメッセージを表示する。
5. タスクの場合、科目と着手予定日を受け取る。
6. 入力されたTodo情報をデータベースに登録する。
7. 登録が成功した場合、追加元の日付の一覧画面へリダイレクトする。

3.2 分岐条件

条件	処理
タスクが選択されている	科目、着手予定日を含めて登録
スケジュールが選択されている	科目、着手予定日なしで登録
タイトルが空	エラーメッセージを表示
入力内容が正常	todo テーブルに登録

3.3 モジュール入出力

モジュール	入力	出力
add.php	タイトル、種別、科目、着手予定日、日付、時刻、メモ	Todo登録結果
db.php	DBファイル	PDO接続

4. 日付別Todo一覧表示

4.1 処理フロー

1. ユーザーがカレンダーから日付を選択する。
2. date.php がURLから日付を受け取る。
3. セッションからログイン中のユーザーを取得する。
4. 指定された日付とユーザーIDを条件に todo テーブルを検索する。
5. 取得したTodoを画面に表示する。
6. 完了済みのTodoは完了状態が分かるように表示する。

4.2 モジュール入出力

モジュール	入力	出力
date.php	日付、セッション	指定日のTodo一覧

モジュール	入力	出力
db.php	DBファイル	PDO接続

5. Todo詳細表示・編集

5.1 処理フロー

1. ユーザーがTodo一覧からTodoを選択する。
2. detail.php がTodo IDを受け取る。
3. Todo IDとログイン中のユーザーIDを条件にTodoを検索する。
4. 該当するTodoの詳細情報を表示する。
5. ユーザーが編集を実行する。
6. edit.php が編集後の入力情報を受け取る。
7. 入力内容を確認する。
8. 入力内容が正常な場合、Todo情報を更新する。
9. 更新後、詳細画面へ移動する。

5.2 分岐条件

条件	処理
Todoが存在する	詳細情報を表示
Todoが存在しない	エラーまたは一覧画面へ移動
入力内容が正常	DBを更新
入力内容に不備がある	エラーメッセージを表示

6. Todo削除

6.1 処理フロー

1. ユーザーがTodo詳細画面で削除を実行する。
2. delete.php がTodo IDを受け取る。
3. Todo IDとログイン中のユーザーIDを確認する。
4. 対象のTodoをデータベースから削除する。
5. 削除後、Todo一覧画面へリダイレクトする。

6.2 分岐条件

条件	処理
対象Todoが存在する	Todoを削除
対象Todoが存在しない	削除処理を行わない

7. 完了状態変更

7.1 処理フロー

1. ユーザーがTodo一覧で完了状態変更を実行する。
2. `complete.php` がTodo IDと完了状態を受け取る。
3. 対象Todoの完了状態を確認する。
4. 完了状態をデータベースに保存する。
5. Todo一覧を再表示する。

7.2 分岐条件

条件	処理
未完了	完了状態に変更
完了済み	未完了状態に変更

8. アプリ内お知らせ

8.1 処理フロー

1. ユーザーがお知らせ画面を開く。
2. `notice.php` がセッションからログイン中のユーザーIDを取得する。
3. 現在の日付と翌日の日付を取得する。
4. `todo` テーブルから、ログイン中のユーザーに関連する未完了のTodoを検索する。
5. 着手予定日が今日、または締切日が今日・明日のTodoを取得する。
6. 取得したTodoの科目の有無を確認する。
7. 科目が登録されている場合は課題のお知らせとして表示する。
8. 科目が登録されていない場合は予定のお知らせとして表示する。
9. 該当するTodoがない場合は、お知らせがないことを表示する。

8.2 分岐条件

条件	お知らせ
未完了かつ着手予定日が今日	今日着手予定の課題があります
未完了かつ締切日が今日	今日締切の課題があります
未完了かつ締切日が明日	明日締切の課題があります
科目が登録されている	課題として表示
科目が登録されていない	予定として表示
条件に該当しない	該当するお知らせなし